

TP4, numpy

1- Soit le système suivant:

$$\begin{cases} v - w = 3 \\ -2u + 4v - w = 1 \\ -2u + 5v - 4w = -2 \end{cases}$$

1- Sur papier, utiliser l'élimination de Gauss pour résoudre le système

2- Utiliser `x = np.linalg.solve(A, B)` pour résoudre le système. Est ce que vous trouvez le même résultat ?

2-

Soit les deux matrices

$$A = \begin{pmatrix} 9 & -6 & 10 \\ -5 & 2 & -5 \\ -12 & 6 & -13 \end{pmatrix} \text{ et } B = \begin{pmatrix} 9 & -6 & 10 \\ -5 & 2 & -5 \\ -12 & 6 & -13 \end{pmatrix}$$

1- Calculer à la main AB .

2- Ecrire une fonction qui retourne le produit scalaire de deux matrices lignes

3- Ecrire une fonction qui prend en argument deux matrices et qui renvoie le produit, en utilisant la fonction du 2-

4- Comparer les temps de calcul du 3 avec `C = np.dot(A, B)` sur des matrices de très grandes tailles (voir tp précédent)

3-

Soit une matrice A de valeurs réels contenant les notes d'étudiants dans différentes matières. Les lignes correspondent aux étudiants de la L1 informatique, par exemple et les colonnes correspondent aux notes dans différentes matières. Par exemple, l'étudiant qui se trouve à la ligne e admet comme note $A[e, m]$ dans la matière qui retrouve à la colonne m .

1 Ecrire une fonction qui renvoie:

- la moyenne et l'écart-type de tous les étudiants dans toutes les matières
- les moyennes et les écart-types de tous les étudiants par matière

2- Ecrire une fonction qui renvoie les indices de la matrice pour lesquelles la moyenne est supérieure à une valeurs donnée.

4-

- 1- Créer une matrice A de taille 5×5
- 2- Calculer son inverse (avec une fonction de numpy)
- 3- Vérifier que $A \times A^{-1} = I_5$

5-

```
import numpy as np

def chercher_pivot(M, l, c):
    pivot = l
    for k in range(l, M.shape[0]):
        if abs(M[k][c]) > abs(M[pivot][c]):
            pivot = k
    return pivot

def echange_lignes(M, i, j):
    M[i], M[j] = M[j].copy(), M[i].copy()

def transvection(M, i, j, c):
    M[j] = M[i, c] * M[j] - M[j, c] * M[i]

def echelonner(A):
    n1 = A.shape[0]
    n2 = A.shape[1]
    l = 0
    for c in range(n2):
        if l == (n1 - 1):
            break
        pivot = chercher_pivot(A, l, c)
        if (A[pivot][c] == 0):
            continue
        if pivot > l:
            echange_lignes(A, l, pivot)
        for k in range(l + 1, n1):
            transvection(A, l, k, c)
        l = l + 1
```

- 1- Ecrire le code ci-dessus et tester le sur les matrices:

```
A = np.array([[1, 2, 1, 1], [2, 4, 2, 2], [3, 6, 3, 4]], dtype=float)
B = A = np.array([[1, 2, 1, 3, 3], [2, 4, 0, 4, 4], [1, 2, 3, 5, 5],
                  [2, 4, 0, 4, 7]], dtype=float)
```

- 2- Modifier le code pour que la fonction `echelonner` renvoie le rang de la matrice. Le rang d'une matrice est le nombre (max) de vecteurs colonnes qui forment une famille libre. C'est aussi le nombre de pivot (non nuls forcément).

- 3- En fait, la famille libre de taille maximale est celle composée des vecteurs colonnes correspondants à des pivots (non nuls) dans la forme échelonnée de la matrice initiale. Modifier la fonction `echelonner` pour qu'elle renvoie une matrice dont les vecteurs colonnes forment la sous famille libre de taille maximale